

What is Hashing?

- Hashing is a technique to uniquely identify a specific object from a group of similar objects.
- It is used in data structures like hash tables, cryptography, digital signatures, etc.
- A hash function maps input data of arbitrary size to fixed-size values.

Example: Simple Modulo Hash Function

Hash Function

$$h(k) = k \pmod{m}$$

- k = key
- m = size of hash table

Example: Simple Modulo Hash Function

Hash Function

$$h(k) = k \mod m$$

- k = key
- m = size of hash table

Numerical Example

Let $m = 7$, and keys: $k = 10, 20, 30, 23$

$$h(10) = 10 \mod 7 = 3$$

$$h(20) = 20 \mod 7 = 6$$

$$h(30) = 30 \mod 7 = 2$$

$$h(23) = 23 \mod 7 = 2 \quad (\text{Collision with } 30)$$

Collision Resolution: Linear Probing

- If collision occurs at index i , try $i + 1, i + 2, \dots$
- Wrap around to 0 if end is reached.

Continuing the Example

$$h(30) = 30 \bmod 7 = 2$$

$$h(23) = 23 \bmod 7 = 2 \quad (\text{Collision})$$

Try index 3: Occupied

Try index 4: Insert at index 4

Hash Function Example 2: Mid-Square Method

Definition

Square the key and extract middle digits.

Example

Key: $k = 123$

$$123^2 = 15129 \Rightarrow \text{Middle digits: } 512 \Rightarrow h(123) = 512$$

Hash Function Example 2: Mid-Square Method

Collision Example

Key 1: $k_1 = 35$

$$35^2 = 1225 \Rightarrow \text{Middle digits: } 22 \Rightarrow h(35) = 22$$

Key 2: $k_2 = 65$

$$65^2 = 4225 \Rightarrow \text{Middle digits: } 22 \Rightarrow h(65) = 22$$

This is a collision since $35 \neq 65$ but $h(35) = h(65)$.

Hash Function Example 3: Folding Method

Definition

Break the key into parts and add them.

Example

Key: $k = 123456 \rightarrow$ Fold as $123 + 456 = 579$

$$h(123456) = 579$$

Hash Function Example: Folding Method Collision

Collision Example

Key 1: $k_1 = 123456$

$$123 + 456 = 579 \Rightarrow h(123456) = 579$$

Key 2: $k_2 = 456123$

$$456 + 123 = 579 \Rightarrow h(456123) = 579$$

Key 3: $k_3 = 222357$

$$222 + 357 = 579 \Rightarrow h(222357) = 579$$

These are collisions because $123456 \neq 456123 \neq 222357$ but all produce $h(k) = 579$.

Note

Any rearrangement of the digit groups that sums to the same total will collide. Also, different groupings like $22 + 23 + 57 = 102$ could collide with other combinations.

Hash Function Example 4: Folding by Shifting

Definition

Alternate parts are reversed before addition.

Example

Key: $k = 123456 \rightarrow$ Fold as $123 + 654 = 777$

$$h(123456) = 777$$

Hash Function Example 5: Multiplication Method

Definition

$$h(k) = \lfloor m(kA \bmod 1) \rfloor$$

Where $A \in (0, 1)$, often $A = 0.618$

Example

Let $k = 1234$, $m = 100$, $A = 0.618$

$$h(1234) = \lfloor 100(1234 \times 0.618 \bmod 1) \rfloor \approx \lfloor 100(0.012) \rfloor = 1$$

Hash Function Example 6: Digit Extraction

Definition

Extract specific digits from the key.

Example

Key: $k = 987654321$, Extract 3rd, 5th, 7th digits: 7, 5, 3

$$h(987654321) = 753$$

Problem Statement 2: Collision Possibility

Problem Statement 2

You are using a digit-extraction hash function that extracts the **1st, 3rd, and 5th digits** from a 7-digit key.

- ① Compute $h(k)$ for the following:
 - $k = 1234567$
 - $k = 1334579$
 - $k = 1534571$
- ② Is there any collision? Justify your answer.
- ③ Suggest a better digit position set (still extracting 3 digits) to minimize collision for 7-digit keys.

Problem Statement 2: Collision Possibility

Solution

1. Extract the 1st, 3rd, and 5th digits for each key:

$$h(1234567) = 1, 3, 5 \Rightarrow h(1234567) = 135$$

$$h(1334579) = 1, 3, 5 \Rightarrow h(1334579) = 135$$

$$h(1534571) = 1, 3, 5 \Rightarrow h(1534571) = 135$$

2. **Collision:** There is a collision since:

$$h(1234567) = h(1334579) = h(1534571) = 135$$

These three keys hash to the same value, which shows a collision.

3. **Suggested Solution:** To minimize collision, you could extract the 2nd, 4th, and 6th digits instead. This will help reduce the chance of multiple keys hashing to the same value, as these positions are more likely to vary.

Hash Function Example 7: Truncation

Definition

Truncate digits of the key to get the hash.

Example

Key: $k = 346781$, Take last 3 digits: $h(k) = 781$

Problem Statement: Truncation Hash Function

Problem Statement

You are using a truncation-based hash function that extracts the **last 4 digits** from a 7-digit key.

- ① Compute $h(k)$ for the following keys:
 - $k = 1234567$
 - $k = 9876543$
 - $k = 5432109$
- ② Is there any collision? Justify your answer.
- ③ What would be the hash result for the key $k = 8453021$?

Problem Statement: Truncation Hash Function

Solution

1. Extract the last 4 digits for each key:

$$h(1234567) = 4567$$

$$h(9876543) = 6543$$

$$h(5432109) = 2109$$

2. **Collision Check:** There is no collision between the hashes of the three keys since:

$$h(1234567) = 4567, \quad h(9876543) = 6543, \quad h(5432109) = 2109$$

All hashes are unique.

3. For the key $k = 8453021$:

$$h(8453021) = 3021$$

Hash Function Example 8: Universal Hashing

Definition

Use random coefficients:

$$h(k) = ((ak + b) \bmod p) \bmod m$$

Example

Let $a = 3, b = 5, p = 97, m = 10, k = 21$

$$h(21) = ((3 \cdot 21 + 5) \bmod 97) \bmod 10 = (68 \bmod 97) \bmod 10 = 8$$

Problem Statement: Universal Hashing

Definition

Use random coefficients:

$$h(k) = ((ak + b) \bmod p) \bmod m$$

Problem Statement

Consider the universal hashing function with the following parameters:

$$a = 7, \quad b = 11, \quad p = 101, \quad m = 12$$

Compute the hash value for the following keys:

- ① $k = 45$
- ② $k = 89$
- ③ $k = 62$

What are the hash values for each key? Is there any collision between the hash values?

Solution: Universal Hashing

Solution

The universal hashing function is:

$$h(k) = ((ak + b) \bmod p) \bmod m$$

With parameters $a = 7, b = 11, p = 101, m = 12$, we compute the hash for each key:

1. For $k = 45$:

$$h(45) = ((7 \cdot 45 + 11) \bmod 101) \bmod 12 = (316 \bmod 101) \bmod 12 = 14 \bmod 12 = 2$$

2. For $k = 89$:

$$h(89) = ((7 \cdot 89 + 11) \bmod 101) \bmod 12 = (634 \bmod 101) \bmod 12 = 28 \bmod 12 = 4$$

3. For $k = 62$:

$$h(62) = ((7 \cdot 62 + 11) \bmod 101) \bmod 12 = (439 \bmod 101) \bmod 12 = 35 \bmod 12 = 11$$

Since all the hash values $h(45) = 2$, $h(89) = 4$, and $h(62) = 11$ are unique, there is **no collision** between the hash values.

Hash Function Example 9: Sum of ASCII Values

Definition

Add ASCII values of characters and mod by table size.

Example

Key = "cat": ASCII = $99 + 97 + 116 = 312$

$$h(\text{"cat"}) = 312 \bmod 10 = 2$$

Problem Statement: Sum of ASCII Values

Problem Statement

Using the sum of ASCII values hashing method, compute the hash value for the following strings, assuming the table size is 10:

- ① Key = "apple"
- ② Key = "banana"
- ③ Key = "grape"

What are the hash values for each key?

Solution: Sum of ASCII Values

Solution

For each key, we will compute the sum of the ASCII values of the characters and then take the modulus by 10.

1. For key "apple":

$$\text{ASCII of "apple"} = 97 + 112 + 112 + 108 + 101 = 530$$

$$h(\text{"apple"}) = 530 \bmod 10 = 0$$

2. For key "banana":

$$\text{ASCII of "banana"} = 98 + 97 + 110 + 97 + 110 + 97 = 609$$

$$h(\text{"banana"}) = 609 \bmod 10 = 9$$

3. For key "grape":

$$\text{ASCII of "grape"} = 103 + 114 + 97 + 112 + 101 = 527$$

$$h(\text{"grape"}) = 527 \bmod 10 = 7$$

So, the hash values for each key are:

$$h(\text{"apple"}) = 0, \quad h(\text{"banana"}) = 9, \quad h(\text{"grape"}) = 7$$

Hash Function Example 10: Polynomial Rolling Hash

Definition

$$h(s) = \left(\sum_{i=0}^{n-1} s[i] \cdot p^i \right) \bmod m$$

Example

String = "abc", ASCII: 97, 98, 99; $p = 31, m = 1000$

$$h(\text{"abc"}) = (97 \cdot 1 + 98 \cdot 31 + 99 \cdot 31^2) \bmod 1000 = 216$$

Problem Statement: Polynomial Rolling Hash

Problem Statement

Using the Polynomial Rolling Hash function, compute the hash value for the following strings with $p = 31$ and $m = 1000$:

- ① String = "xyz"
- ② String = "hello"
- ③ String = "world"

What are the hash values for each string?

Solution: Polynomial Rolling Hash

Solution

The Polynomial Rolling Hash function is:

$$h(s) = \left(\sum_{i=0}^{n-1} s[i] \cdot p^i \right) \mod m$$

For each string, compute the hash using the ASCII values of the characters, $p = 31$, and $m = 1000$.

1. For string "xyz":

ASCII of "xyz" = 120, 121, 122

$$h("xyz") = (120 \cdot 1 + 121 \cdot 31 + 122 \cdot 31^2) \mod 1000$$

$$h("xyz") = (120 + 3751 + 117086) \mod 1000 = 121957 \mod 1000 = 957$$

2. For string "hello":

ASCII of "hello" = 104, 101, 108, 108, 111

$$h("hello") = (104 \cdot 1 + 101 \cdot 31 + 108 \cdot 31^2 + 108 \cdot 31^3 + 111 \cdot 31^4) \mod 1000$$

$$h("hello") = (104 + 3131 + 104528 + 3248428 + 9948721) \mod 1000 = 13344212 \mod 1000 = 212$$

3. For string "world":

ASCII of "world" = 119, 111, 114, 108, 100

$$h("world") = (119 \cdot 1 + 111 \cdot 31 + 114 \cdot 31^2 + 108 \cdot 31^3 + 100 \cdot 31^4) \mod 1000$$

$$h("world") = (119 + 3441 + 109215 + 3252088 + 9125000) \mod 1000 = 12540663 \mod 1000 = 663$$

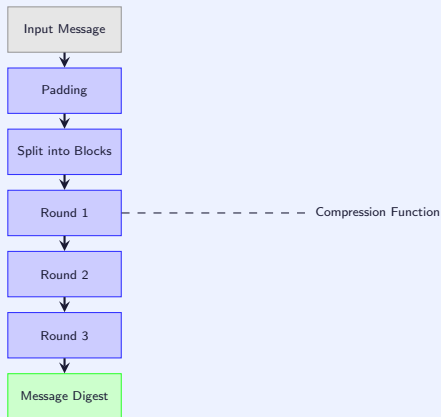
Solution: Polynomial Rolling Hash

Solution

So, the hash values for each string are:

$$h(\text{"xyz"}) = 957, \quad h(\text{"hello"}) = 212, \quad h(\text{"world"}) = 663$$

MD1 Hashing Algorithm Diagram



MD1 Hashing Rounds Overview

Round 1: Initial Mixing

- Input: One block of the padded message.
- Operations:
 - Split block into words (e.g., $M[0], M[1], M[2], M[3]$).
 - Bitwise XOR and modular additions with internal state (A, B, C, D) .
 - Possible left/right bit rotations.
- Example:

$$A = A + M[0], \quad B = B \oplus M[1], \quad C = C + (M[2] \ll 3), \quad D = D \oplus (M[3] \gg 2)$$

Round 2: Permutation & Strengthening

- Mix internal state using additional operations and round constants.
- Example:

$$A = A + (B \oplus C), \quad B = \text{ROL}(B + D, 5), \quad C = C + A, \quad D = \text{ROR}(D \oplus B, 3)$$

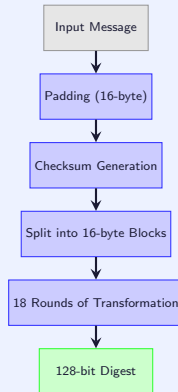
MD1 Hashing Rounds Overview

Round 3: Final Diffusion

- Ensure small input change affects the full digest (avalanche effect).
- Example:

$$A = A \oplus C, \quad B = (B + A) \bmod 2^{32}, \quad C = C \oplus D, \quad D = D + A$$

MD2 Hashing Algorithm Diagram



MD2 Hashing: Preprocessing

Padding

- Message is padded to a multiple of 16 bytes.
- Padding byte value is equal to the number of padding bytes.
- E.g., if 10 bytes of padding needed: 0x0A 0x0A ... 0x0A.

Checksum Generation

- A 16-byte checksum is appended to the message after padding.
- Computed using XOR and a substitution table (S-table).
- Ensures integrity and sensitivity to message changes.

MD2 Hashing: Main Transformation

18 Rounds of Transformation

- Data block (16 bytes) and internal 48-byte buffer are updated.
- Uses substitution values from a fixed 256-byte S-table.
- Each round:
 - Updates each byte of the buffer using XOR with S-table entry.
 - Introduces dependency across all positions and rounds.
- Ensures diffusion and non-linearity.

MD2 Final Output

Digest

- After all transformations, the first 16 bytes of the internal buffer are used as the final hash.
- Output is a 128-bit (16-byte) hash value.

$h(M)$ = first 16 bytes of final internal state

Example Output (Hex)

8350e5a3e24c153df2275c9f80692773

33

- Secure hash functions like MD5, SHA-256
- Irreversible and collision-resistant
- Applications:
 - Password hashing
 - Blockchain
 - Digital Signatures

Example: SHA-256 Hash

Hash of "Hello"

SHA256("Hello") =

185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969